CSD493 PROJECT-1 · SECURITY RESEARCH STUDY

Behavioural-Diff Detection of Malicious Browser-Extension Updates

Feasibility, Security Gap & Novelty — an Evidence-Based Study

Team: Bind Pratap Singh (2310110084) · Suhani Deepak Agrawat (2310110717)

Advisor: Dr. Sweta Mishra · **Specialisation:** Cybersecurity

Programme: B.Tech CSE · Shiv Nadar Institution of Eminence

Prepared: September 2026 · **Project:** extdrift

> **Scope.** This study answers, from peer-reviewed literature and vendor documentation rather than from our own optimism, the three questions raised by the advisor: (1) does an extension's language/runtime affect its security, (2) can our system realistically protect an end user, and (3) how do we differ from existing browser/store defences. It was written to *test*, not assume, the project's novelty and feasibility — and two findings reshape our positioning.

Contents

0 Two corrections this research forces on our own materials

1 Executive Summary

2 Extension Security Architecture

3 Language / Runtime Security (Q1)

4 Existing Browser / Store Defences (Q3)

5 Real-World Incidents — where security actually failed

6 End-User Feasibility (Q2)

7 Research Gap & Novelty

8 Capability Comparison Table

9 Recommended (Narrowed) Research Problem

10 Suggested Experiment to Prove Added Value

11 Our Prototype — current output

12 What We Should Tell Our Professor

13 References

FACT established / vendor-documented **FINDING** from a cited paper/report **ASSUMPTION** reasonable, unverified

OURS our interpretation/design

Figures read at search-level from papers are marked *verify in source* — confirm the exact number in the PDF before quoting it to an examiner.

0. Two corrections this research forces on our own materials

> **⚠ 1 — The core idea is partly taken; we must reposition, not retreat.**
> A top-tier paper already does *update-delta* detection of malicious extensions: **Pantelaios, Nikiforakis, Kapravelos, "You've Changed: Detecting Malicious Browser Extensions through their Update Deltas," ACM CCS 2020** (DOI · PDF). It analyses **922,684 versions over six years** and finds **143 malicious extensions in 21 clusters** *(verify in source)*. **But it is STATIC code-delta analysis** — it locates the *code* added between versions and encodes the delta by the APIs it abuses; its first stage even triggers on anomalous *ratings*. It is admitted to be **obfuscation-evadable**. So "diff two versions" is **not** our novelty — our wedge is "diff two versions **dynamically, by behaviour, with deterministic inputs**," which is obfuscation-resistant by construction (see §7).

> **⚠ 2 — Our "Karami (NDSS 2021)" citation is misattributed; fix it before the meeting.**
> The real paper — **Karami, Ilia, Polakis, "Awakening the Web's Sleeper Agents: Misusing Service Workers for Privacy Leakage," NDSS 2021** — is about the **Service Worker *web* API abused by *websites***, **not** Manifest V3 *extension* background service workers. Citing it for "MV3 extension service workers leak without a page" is a mischaracterisation an examiner could catch. Recharacterise it accurately, or cite the Chrome MV3 architecture docs plus a MV3-security paper (§3). Karami also authored *Carnus* (NDSS 2020) on extension *fingerprinting* — do not conflate the two.

Neither is fatal. Both make us look **more** rigorous if *we* raise them first.

1. Executive Summary

- FINDING **Extension insecurity is not a language/runtime problem.** Extensions are JavaScript/HTML/CSS in a memory-safe engine; the dominant risks are **permissions, host access, powerful browser APIs, third-party/supply-chain code, and human/account compromise** — not memory bugs or the choice of JS vs WASM. *Implication: the "language-aware analysis" angle in our original pitch is weak and should be dropped.* (§3)
- FINDING **For an ordinary end user, prevention-before-execution is largely infeasible for an external tool.** Updates are silent and automatic; the browser gives outside tools no pre-execution veto. Realistic value is at the **store / enterprise / researcher** tier and as **fast detection + response**, not consumer prevention. (§6)
- FACT **Browsers/stores already do a lot** — package signing, manifest/permission validation, automated static + manual review (updates included), malware/reputation, isolation/CSP, and MV3's remote-code ban. FINDING **But these are bypassable** by obfuscation, dormant payloads, staged rollouts, and — above all — **compromised-legitimate extensions** whose malicious *update* passes review. Cyberhaven (Dec 2024) was an **OAuth/account compromise**, not an extension-analysis failure. (§4–§5)
- OURS **The honest, defensible gap:** *deterministic, dynamic, behavioural diffing of consecutive versions to catch security-relevant behavioural change introduced by an update — especially in compromised-legitimate extensions — with explainable output.* The static version (You've Changed) exists; the dynamic, determinism-anchored, obfuscation-resistant version is open. (§7)

2. Extension Security Architecture (dev → store → install → runtime)

FACT **Build & languages.** Modern extensions (Chrome/Chromium, Edge, Firefox, Safari) are **web apps**: JavaScript/TypeScript, HTML, CSS, JSON manifest; optionally WebAssembly and — via **native messaging** — a separately-installed native host. They run as **content scripts** (injected into pages), a **background service worker** (MV3) or page (MV2), and optional UI pages. Capabilities are gated by declared **permissions** and **host permissions** (Chrome MV3 docs).

FACT **Distribution & update.** Extensions ship through a **store** (Chrome Web Store, Firefox AMO, Edge Add-ons, Safari via an Xcode-built app). The browser **auto-updates** them silently; the user is not asked to re-approve unchanged permissions, and some permission additions raise no clear prompt — **"silent permission creep"** (Pluto Security).

FACT **Store review (Chrome).** Every submission — **including updates** — goes through **automated analysis then human review**: malware-signature scanning, obfuscation and dangerous-API heuristics, policy checks, manifest validity, install testing (review-process docs).

FACT **Runtime model.** Content scripts run in an **isolated world**; the extension is constrained by **CSP** and its permission set; **MV3 forbids remotely-hosted code** (improve-security docs). Once installed, an extension with broad host access effectively **sits in the middle of the user's web traffic** (Island).

3. Language / Runtime Security — does the language matter? (Q1)

Short answer: FINDING **No, not materially.** The evidence points to *permissions, APIs, logic, dependencies, and human/supply-chain factors* as the real sources of risk — not the language or runtime.

- FACT **Memory safety is a non-issue.** Extension code runs in the browser's memory-safe JS engine; classic C/C++ bugs (buffer overflow, use-after-free) do not arise from extension JS. That whole vulnerability class is absent by construction.
- FINDING **The real vectors are capability- and logic-level:** over-broad host permissions, abuse of powerful APIs (`cookies` , `webRequest` / `declarativeNetRequest` , `scripting` , `tabs`), insecure message passing, DOM injection, and **dynamic code execution**, plus **third-party/supply-chain** code (Island, Aikido).
- FINDING **JS-specific weaknesses (e.g., prototype pollution) are logic bugs, not runtime faults** — they come from unsafe object handling in code/deps.
- FINDING **WebAssembly / native components add analysis *difficulty*, not a dominant attack class.** WASM and obfuscation mainly make code harder to inspect *statically*; native messaging widens the trust boundary. Their real relevance to us: they **defeat static analysis** — an argument *for* dynamic analysis, not for "language-aware" analysis.
- FINDING **MV3 reduced the runtime surface** — removed remote code, deprecated **~87.8% of vulnerable APIs** *(verify in source:* Manifest V3 Unveiled, arXiv 2404.08310*)* — yet **~29.8% of analysed malicious extensions remained operational after V3 conversion** and **~56% adapted** via web-accessible resources *(verify in source)*. The remote-code ban is **circumventable** by shipping configuration/rules instead of code (Palant, 2025).

> **Consequence for difficulty & design** OURS
>
> Because risk lives in *behaviour and capability*, not in the language, a reliable analyser must observe **what the extension does at runtime with its permissions**. This is *why* static analysis is hard (obfuscation/WASM/dynamic code hide intent) and *why* our dynamic, behavioural stance is the right lever. It also means we should **not** claim "language-aware detection" as a contribution — the evidence doesn't support it.

4. Existing Browser / Store Defences — what they catch, where they stop (Q3)

Mechanism	What it does	Where it stops (FINDING)
Package signing / CRX validation	Integrity + provenance of the package	Doesn't judge **behaviour**; a signed update from a **compromised** account is still trusted
Manifest & permission validation	Enforces declared capabilities	Declaration ≠ intent; **silent permission creep** on update is often unprompted
Automated static review	Malware signatures, obfuscation & dangerous-API heuristics	**Bypassable** by obfuscation, WASM, dynamic code, **dormant/condition-gated payloads**
Human review	Manual code/policy check	Not exhaustive; can't run every trigger path; **post-review updates** & staged rollouts evade
Malware/reputation + blocklist	Flags known-bad, revokes	**Reactive** — needs prior knowledge; zero-day updates slip until reported
Isolation / CSP / MV3 no-remote-code	Shrinks runtime surface	**~30% of malicious extensions still function post-MV3**; rules/config channels evade
Store review applies to updates	Same pipeline for updates	Same weaknesses; **trusted-then-malicious** update is the blind spot

FINDING **Cross-browser differences. Firefox AMO** signs every add-on after automated validation, with manual review at any time (Extension Workshop); a long-standing critique is that automated scanning is limited (Stillman, 2015). **Safari** ships extensions **inside a signed macOS/iOS app built with Xcode** — a different artifact, toolchain and review queue, and a macOS-only, structurally different target (why we exclude it).

> *The pattern: existing defences are strong on **integrity, declaration, and known-bad**, and weak on **novel behavioural change on update** — especially obfuscated/dormant payloads and compromised-legitimate extensions.*

5. Real-World Incidents — where exactly did security fail?

Incident	What happened	Failure locus (FINDING)	Detected
Cyberhaven + campaign (Dec 2024)	OAuth-phishing of dev accounts ("Privacy Policy Extensions" app) → malicious updates to **~35 extensions / 2.6M users** exfiltrating cookies/session tokens	**Account/OAuth compromise + supply-chain**, *not* an extension-analysis failure; the update then shipped through the normal pipeline	**After** exposure — live ~24h+
The Great Suspender (2021)	Extension (2M+ users) **sold to unknown party**, which added tracking/remote-code	**Ownership transfer + trust hand-off**; malicious code arrived via update	After; later removed
Nano Adblocker/Defender (2020)	Devs **sold rights**; new owner added data-collection	**Ownership transfer / acquire-and-subvert**	After user reports
DataSpii (2019)	Several extensions exfiltrated browsing/PII at scale	**Permission abuse + undisclosed collection**	After investigation

Sources: Sekoia, SecurityWeek, The Register, LWN.

> **The distinction the professor drew is correct and central** OURS
> *Developer-account/OAuth compromise ≠ extension-analysis failure*. In Cyberhaven no analyser was defeated at upload time — the **account** was taken and the update **passed review**. This is exactly the case a behavioural *update-diff* addresses: it doesn't care *how* the bad code got in (phishing, sale, insider); it asks **"does this new version suddenly *do* something the trusted one didn't?"** — the one question that catches trusted-then-malicious regardless of entry vector.

6. End-User Feasibility — protect the user before the update runs? (Q2)

> **Honest conclusion (OURS)**
> For an ordinary end user, an external tool generally **cannot prevent** a malicious update from executing — it can, at best, **detect fast and respond**. Real prevention lives one tier up (store / enterprise / researcher).

- FACT **Updates are silent and automatic.** The user never sees the new version before it runs; there is no natural interception point for a third party on a consumer machine.
- FACT **The browser gives outside tools little pre-execution power.** An extension can't reliably inspect or veto a *sibling* extension's update; an OS agent sees files but not browser-internal behaviour without deep, privileged instrumentation.

Deployment model	Pre-exec prevention?	Visibility	Notes
Browser-integrated (vendor)	Yes	Full	Only the vendor can do this; not us
Store-side scanning (pre-distribution)	**Yes** — before users get it	Full (all versions)	The natural home for update-diffing; "perfect fidelity"
Enterprise policy / fleet monitoring	Partial (allowlist + fast pull)	High	Realistic, valuable, deployable

Deployment model	Pre-exec prevention?	Visibility	Notes
Endpoint / EDR agent	Detect > prevent	Medium	Sees files/network, not browser internals
A security *extension*	No (can't veto siblings)	Low	Weakest option; avoid claiming consumer prevention
Researcher/analyst tool (ours today)	Detect (offline)	Full (lab)	Where our prototype sits; strongest for evidence

> *Say plainly to the professor: "We can't stand between Google's silent auto-update and one user's browser; we can give whoever governs the update channel — a store reviewer, an enterprise, a researcher — an automatic behavioural re-check of every new version, which is exactly the check nobody runs today."*

7. Research Gap & Novelty — classified honestly

Solved well FACT **:** package signing/integrity; permission declaration & isolation; CSP; MV3 remote-code ban; known-malware/reputation blocklisting; store review pipeline.

Partially solved FINDING **:** static review of updates (bypassable); permission-change surfacing (silent creep); **static update-delta detection** — solved by *You've Changed* (CCS 2020), but statically and thus obfuscation-limited.

Weakly solved FINDING **:** scalable **dynamic behavioural** analysis of extensions (Hulk 2014, ExtPrivA S&P 2023, Arcanum USENIX 2024, Fakeium 2024 exist, but are **single-version**); detecting **dormant/condition-gated** payloads; explainable verdicts.

Open / unsolved OURS **: dynamic, behavioural, cross-version diffing with record/replay determinism** — scoring the behavioural *change* of an update obfuscation-resistantly, targeting **compromised-legitimate** extensions, with **explainable** output. No work we found occupies this exact cell.

> **The novelty wedge, stated defensibly**
> Prior update-aware detection (You've Changed, CCS 2020) is **static code-delta** and, by the authors' own account, evadable by obfuscation. Prior dynamic behavioural analysis (Hulk, ExtPrivA, Arcanum) is **single-version**. **extdrift sits at the intersection: behavioural update-diffing made reliable by record/replay determinism** — obfuscation-resistant (we observe effects, not code) and false-positive-resistant (both versions see byte-identical input, so the diff is the extension, not web noise).

OURS **What to drop from the original idea:** "language-aware analysis" (§3 shows it doesn't matter) and any claim of **consumer pre-execution prevention** (§6 shows it's infeasible for an external tool).

8. Capability Comparison Table

"~" = partial; claims limited to what evidence supports.

Capability	Chrome/Chromium	Firefox	Edge	Safari	extdrift (ours)
Static / package checks	Yes	Yes	Yes	Yes (app-signed)	~ (manifest/perm diff)

Capability	Chrome/Chromium	Firefox	Edge	Safari	extdrift (ours)
Permission checks	Yes	Yes	Yes	Yes	Yes (+ escalation diff)
Malware / reputation	Yes	Yes	Yes	Yes	No (not our aim)
Update validation	Yes (integrity)	Yes	Yes	Yes	**Yes — behavioural**
Runtime protection (isolation/CSP/MV3)	Yes	Yes	Yes	Yes	No (analyse, not enforce)
Pre-installation protection	Yes (store)	Yes (store)	Yes	Yes	~ (if store/enterprise-deployed)
Runtime behavioural detection	~ limited	~	~	~	**Yes**
Update-to-update *behavioural* comparison	**No**	**No**	**No**	**No**	**Yes (our core)**
Protection from **compromised-legit** extensions	~ reactive	~	~	~	**Yes (design target)**
Obfuscation-resistant update analysis	~ (static, evadable)	~	~	~	**Yes (dynamic)**
Explainable per-verdict output	No	No	No	No	**Yes (rules + SHAP)**

Static code-delta update analysis (*You've Changed*) is a research method, not a browser feature, so it isn't a column; the row that matters against it is **obfuscation-resistant update analysis**, where our dynamic approach is the differentiator.

9. Recommended (Narrowed) Research Problem

> *Given two consecutive versions of the same browser extension, detect — dynamically and obfuscation-resistantly — whether the update introduces security-relevant behavioural change (new data access, new exfiltration, new capability use), with an explainable verdict, using record/replay determinism to suppress false positives; targeted at compromised-legitimate extensions and positioned for store-side / enterprise / researcher deployment rather than consumer pre-execution prevention.*

Why this is the strongest version: it (a) **concedes** the prior static update-delta work and claims only the open dynamic/deterministic cell; (b) **drops** the two indefensible claims (language-aware; consumer prevention); (c) foregrounds **determinism + explainability**, which address the field's real bottleneck (false positives) rather than chasing model accuracy.

10. Suggested Experiment to Prove Added Value

> **The one experiment that proves we add something over the closest prior work**
> **Head-to-head on obfuscation.** Take update pairs where the malicious `v2` carries an **obfuscated / dynamically-decoded / condition-gated** payload. Show that a **static code-delta** approach (You've-Changed style) **misses or mis-scores** it, while our **dynamic behavioural diff** still flags it because the payload's *runtime effect* (cookie read → new-host POST from the service worker) is observable regardless of how the code is written.

Plus two supporting results we can already produce:

1. **False-positive rate on real benign updates** (GitHub-release pairs of open-source extensions) — the metric that actually matters — **with vs. without record/replay determinism** (the ablation), showing determinism drives false positives toward zero.
2. **Detection + ground truth** on the synthetic weaponisation corpus and the reconstructed **Cyberhaven** pair, each with an explainable verdict.

11. Our Prototype — current output

The pipeline already runs end-to-end on a real browser (loads two versions, drives an identical scripted session against deterministic pages, records four channels — network / DOM / storage / API incl. the service worker — diffs, scores, and explains). **~3,100 lines of Python + ~300 of instrumentation JS; 72 automated tests, all green** (2 launch a real Chromium). Representative output:

Weaponised update (Cyberhaven-style payload actually executes)	`MALICIOUS · 0.99 · leak confirmed`
Gray-zone update (scraper batches contacts to a new host)	`SUSPICIOUS · 0.68`
Benign feature update (reading-time badge)	`BENIGN · 0.25 · no leak`

The verdict is cross-checked against a fake collector's own log (independent ground truth of what actually left the browser). Reproduce: `python scripts/demo.py` (live) or `python scripts/demo_offline.py` (no browser).

12. What We Should Tell Our Professor

1. **Why is this problem difficult?** Malice is **behavioural and can be hidden** — obfuscated, dynamically decoded, or condition-gated — so static inspection is evadable and dynamic inspection must *trigger* the behaviour and separate it from web noise. Scarce labelled data and silent auto-updates make it harder still.
2. **What makes extension security technically challenging?** Extensions are **trusted, auto-updating code with mid-traffic privilege**; trust is granted once and never re-checked; the malicious change usually arrives **after** all the checks, via an update.
3. **Does language/runtime choice materially matter? No.** Extensions are memory-safe JS; risk comes from permissions, APIs, logic, dependencies, and account/supply-chain compromise. (So we drop the "language-aware" angle.)
4. **Can the end user be protected before a malicious update executes? Not by an external tool, realistically.** Prevention belongs to the vendor/store; our value is **store/enterprise/researcher re-verification** and **fast detection**.
5. **What do browsers already check?** Signing/integrity, manifest/permissions, automated + manual review (updates included), malware/reputation, isolation/CSP, MV3 no-remote-code.
6. **Where do those defences actually fail?** Obfuscated/dormant payloads, silent permission creep, and **compromised-legitimate extensions** whose malicious update passes review — as in Cyberhaven, an **OAuth/account** compromise, not an analysis failure.
7. **How can our approach be different? Dynamic, behavioural, cross-version diffing with record/replay determinism** — obfuscation-resistant and false-positive-resistant, with explainable verdicts. The open cell next to static update-delta (You've Changed) and single-version dynamic work (Hulk, ExtPrivA).

8. **What should we change/narrow?** Drop language-aware analysis and consumer pre-execution prevention; reposition as a store/enterprise/researcher method; foreground determinism + explainability; and **cite You've Changed explicitly** as prior work we improve on dynamically.
9. **Strongest research problem to pursue?** The narrowed statement in §9.
10. **What experiment proves we add value?** The **obfuscation head-to-head** (§10) plus the **determinism ablation** on real benign updates.

13. References

Closest prior work / core papers

Pantelaios, Nikiforakis, Kapravelos. *You've Changed: Detecting Malicious Browser Extensions through their Update Deltas*. ACM CCS 2020. dl.acm.org/doi/10.1145/3372297.3423343 · PDF

Bui, Tang, Shin. *Detection of Inconsistencies in Privacy Practices of Browser Extensions (ExtPrivA)*. IEEE S&P 2023.

Kapravelos, Grier, Chachra, Kruegel, Vigna, Paxson. *Hulk: Eliciting Malicious Behavior in Browser Extensions*. USENIX Security 2014.

Bandhakavi, King, Madhusudan, Winslett. *VEX: Vetting Browser Extensions for Security Vulnerabilities*. USENIX Security 2010.

Karami, Ilia, Polakis. *Awakening the Web's Sleeper Agents: Misusing Service Workers for Privacy Leakage*. NDSS 2021. (Web service workers, not extension MV3 — see §0.)

Karami, Ilia, Solomos, Polakis. *Carnus: Exploring the Privacy Threats of Browser Extension Fingerprinting*. NDSS 2020. PDF

Platform / MV3 / defences

Chrome for Developers. *What is Manifest V3 · Improve extension security · Review process*. what-is-mv3 · improve-security · review-process

Manifest V3 Unveiled: Navigating the New Era of Browser Extensions. arXiv 2404.08310, 2024. arxiv.org/pdf/2404.08310

Palant. *Malicious extensions circumvent Google's remote code ban*. 2025. palant.info

Mozilla. *Add-ons signing & distribution*. extensionworkshop.com · Stillman, *Firefox Extension Scanning is Security Theater*, 2015. danstillman.com

Recent dynamic / analysis work

Khandelwal, Nayak, Fernandes, Fawaz. *Experimental Security Analysis of Sensitive Data Access by Browser Extensions*. ACM WWW 2024. dl.acm.org/doi/10.1145/3589334.3645683

Xie et al. *Arcanum: Detecting and Evaluating the Privacy Risks of Browser Extensions*. USENIX Security 2024. PDF

CodeX: Contextual Flow Tracking for Browser Extensions. CODASPY 2025. dl.acm.org/doi/10.1145/3714393.3726495

Fakeium: A Dynamic Execution Environment for JavaScript Program Analysis. arXiv 2410.20862, 2024. arxiv.org/pdf/2410.20862

A Study on Malicious Browser Extensions in 2025. arXiv 2503.04292. arxiv.org/pdf/2503.04292

Incidents / supply chain

Sekoia. *Targeted supply chain attack against Chrome browser extensions*. 2025. sekoia.io

SecurityWeek. *Cyberhaven Chrome Extension Hack Linked to Widening Supply Chain Campaign*. securityweek.com

The Register. *The Great Suspender*. 2021. theregister.com

LWN. *Malware in open-source web extensions (Nano Adblocker)*. lwn.net/Articles/846272

Pluto Security. *Chrome Extension Supply Chain Attacks: silent permission creep*. pluto.security

---

Prepared for the CSD493 progress meeting · extdrift · This study was written to test, not assume, the project's novelty and feasibility. Tag key: FACT = established/vendor-documented · FINDING = from a cited source · ASSUMPTION = reasonable but unverified ·

OURS = our interpretation.

CSD493 · EXTENDED RESEARCH · PART II

Complete Faculty Validation of a Browser-Extension Update-Security Project

Problem recurrence, difficulty, taxonomy, recovery, model & dataset strategy — evidence-based

Team: Bind Pratap Singh (2310110084) · Suhani Deepak Agrawat (2310110717)

Advisor: Dr. Sweta Mishra · **Project:** extdrift · B.Tech CSE, Shiv Nadar IoE

Companion to: Part I — Feasibility & Novelty Study · **Prepared:** September 2026

Scope. This document answers twelve additional validation requirements the faculty raised before implementation: that the problem recurs beyond Cyberhaven; why it is technically hard; a full vulnerability taxonomy; what is recoverable after a malicious update; a precise ML task; an *evidence-based* model choice (not a pre-assumed XGBoost); realistic datasets; honest handling of unknown threats; the static/dynamic/honeytrap question; and a leakage-free evaluation. It was written to test the project, and it concludes with a **narrow-and-reframe** recommendation, not a rubber stamp.

Contents

1 Problem Validation Beyond Cyberhaven
2 What Makes This Difficult — prior-art status
3 Vulnerability Taxonomy (beyond code injection)
4 After a Malicious Update — Detect → Recover
5 Model Selection — evidence-based, not assumed
6 What Exactly Will the Model Predict?
7 Dataset Strategy
8 Unknown / Hidden Threats — classification vs discovery
9 Static, Dynamic & Honeytrap Pages
10 Evaluation Methodology
11 Final Synthesis (10 points)
12 Complete Faculty Validation (12 answers)
13 Additional References

FACT established / vendor-documented **FINDING** from a cited source **ASSUMPTION** reasonable, unverified

OURS our interpretation/design

Headline for the faculty

The problem is **recurring** (a decade of incidents, tens of millions of users), the task is **genuinely hard** (obfuscation, dynamic behaviour, data scarcity — each only partially solved), and the honest project is **narrowed and reframed**, not rejected: *deterministic dynamic behavioural update-diffing, store/enterprise/researcher-positioned, with an experimentally-chosen model.*

1. Problem Validation Beyond Cyberhaven — recurring, not a one-off

FINDING Malicious/compromised/subverted extensions are a **decade-long, repeating** pattern across independent actors and entry vectors:

Incident (year)	What happened	Entry vector	Users	Why defences failed	Diff catch?
DataSpii (2019)	8+ exts exfiltrated history/PII	Permission abuse	~4M	Runtime exfil unmonitored	Partly
Nano Adblocker (2020)	Sold; data collection added	Ownership transfer	~300k	Trust not re-checked after sale	**Yes**
Great Suspender (2021)	Sold; tracking + remote code	Ownership transfer	~2M	Update passed; no behaviour re-check	**Yes**
Palant "SeaSearch" (2023)	Hidden JS-injection API in 34 exts	Bad code in updates/origin	**~87M installs**	Obfuscated, server-triggered; static missed	**Yes** (update variants)
Cyberhaven + campaign (2024)	Cookie/token theft via update	OAuth phishing of devs	**~2.6M / ~35 exts**	Account compromise; update passed review	**Yes**
Chrome+Edge spyware (2025)	18–19 exts spying/hijacking	Fake tools / subverted	"millions"	Long-dormant trusted listings	Partly/Yes
Crypto stealers (2025–26)	Popular exts weaponised for wallet theft	Subverted popular exts	~80k (one)	Targeted, condition-gated	**Yes**

Sources: BleepingComputer (75M), Kaspersky (87M), Sekoia (Cyberhaven), Malwarebytes (2025), Infosecurity (18 exts), chrome-mal-ids.

> **Conclusion** OURS
>
> Two sub-patterns recur independently — **supply-chain/account compromise** (Cyberhaven) and **ownership-transfer / acquire-and-subvert** (Great Suspender, Nano) — and **both express as a behavioural change between a trusted version and its successor**, exactly what we score. Palant's campaigns show the **obfuscation/dormancy** that defeats static review, motivating our dynamic stance. The project is **not built around one incident**.

2. What Makes This Difficult — challenge-by-challenge

Challenge	Prior-art status	Notes & our exposure
Obfuscation / minification	**Partial**	Defeats static (VEX, You've-Changed admit this). Dynamic sidesteps it — **our advantage**.
Dynamic / condition-gated behaviour	**Partial**	Hulk elicits via HoneyPages; coverage never complete; server-gated payloads can stay dormant.
Run-to-run non-determinism	**Weakly solved**	Record/replay determinism (ours) is the mitigation — **our core contribution**.
Third-party deps / supply chain	Partial	Behavioural diff catches the *effect* regardless of source.
Large permission sets / browser APIs	Partial	Declaration ≠ behaviour; MV2/MV3 + `chrome.*` / `browser.*` normalisation needed.
Static vs dynamic trade-off	Known	Each alone insufficient; hybrid is consensus.
False positives / negatives	**Open (the bottleneck)**	2025 "It's not Easy": 98% lab, >1,000 wild flags, only 68 real. Determinism + diff target FPs.
Previously-unknown payloads	**Open**	Supervised classifies *known*; novelty needs anomaly methods (§8).

Challenge	Prior-art status	Notes & our exposure
Runtime depth (MV3 service worker)	Partial	SW exfiltration is a page-centric blind spot; we instrument it.
Scalability	Solved (engineering)	Hulk did ~48k via parallel automation; not a research blocker.
Dataset scarcity (update pairs)	**Open**	No public labelled update-pair set → synthetic-train + real-validate (§7).

> *The concentration of **partial/open** rows — determinism, false positives, unknown payloads, update-pair data — is what makes this a **research** project, not an implementation task. We attack **determinism + false positives on the update-diff**, the open cell nearest our closest prior work.*

3. Vulnerability Taxonomy (beyond code injection)

OURS synthesised from the literature. **S**=static-detectable, **D**=needs dynamic, **ML?**=learning realistically helps on our features. ✓ yes · ~ partial · ✗ no.

Vulnerability class	How it occurs	Impact	S	D	ML?	Relevance
1 Malicious update / supply-chain	Trusted ext updated with bad code	Mass theft	~	✓	✓	**Core**
2 Credential / cookie / token theft	Read login fields / `cookies.getAll` → exfil	Account takeover	~	✓	✓	**Core**
3 Data exfiltration	Read PII → outbound to new host	Privacy breach	~	✓	✓	**Core**
4 Excessive / escalating permissions	Update adds `<all_urls>`, `cookies`	Wider capability	✓	~	✓	Static signal
5 MV3 service-worker abuse	Background beaconing, no page	Stealth exfil	✗	✓	✓	**Core**
6 Browser-API misuse	`webRequest` / `DNR` / `scripting`	Tamper, inject	~	✓	✓	Core-adjacent
7 DOM injection & UXSS	Inject nodes / bypass isolation	Page hijack, XSS	~	✓	~	Partial
8 Insecure / cross-extension messaging	Unvalidated `onMessage`, `externally_connectable`	Privilege bridge	✓	~	~	Static signal
9 Origin-validation failure	Content script trusts page data	Escalation	✓	~	✗	Secondary
10 Insecure native messaging	Talks to native host	Sandbox escape	✓	~	~	Out of core
11 CSP / policy weakening	Remote code (MV2) / rules channel (MV3)	RCE-like	✓	~	~	Manifest diff
12 Affiliate / ad / search hijack	DNR redirects, script inject	Fraud, tamper	~	✓	✓	Behavioural
13 Fingerprinting / privacy	Probe environment, track	Deanonymise	~	✓	~	Secondary
14 Logic flaws / vulnerable benign (VEX)	Exploitable flow in benign code	Web-triggered escalation	✓	~	✗	**Out of scope**

> *extdrift centres on classes **1–6 and 12** — the behavioural/capability classes that are the actual payloads in §1 and are best seen dynamically. Classes 8–11 and 14 (static, code-vulnerability flavoured) are **explicitly out of scope** — conflating them is the over-broad "vulnerability detection" objective to avoid (§6).*

4. After a Malicious Update — Detect → Block → Disable → Rollback → Recover

FACT Five different capabilities, held by different parties:

Stage	Chrome/Chromium	Firefox	End user?	Notes
Detect	Google review/telemetry; Safety Check	AMO review + blocklist	No (limited)	Mostly vendor-side, often **after** exposure
Block (new installs)	CWS takedown + blocklist	AMO blocklist	No	Stops further spread, not those already updated
Disable (installed)	**Remote kill-switch** (malware flag); enterprise blocklist	Blocklist auto-disables	Partial	Malware-flagged ext can't be re-enabled by user
Rollback (prev version)	**Not for users**; enterprise version-pin	No user rollback	**No**	Stores hold all versions & *could* revert; users cannot
Recover (undo damage)	Out of browser scope	—	**No**	Stolen tokens already out → rotate creds, revoke sessions

Blunt conclusions

Prevention/rollback for an individual user is essentially unavailable — updates are silent, there is no "revert extension" button, and by the time a malicious update runs, **exfiltration may already be complete** (irreversible at the browser layer; only service-side credential rotation helps). Disable/block exist but are **vendor/enterprise-controlled and usually post-exposure**. **This is exactly why our value is early detection at the store/enterprise/researcher tier — before propagation — not consumer recovery.**

5. Model Selection — evidence-based, not "XGBoost because everyone uses it"

FINDING Published extension detectors are dominated by **classic tabular ML on static/metadata features** (SVM, RF, KNN, XGBoost; combined ML+feature-engineering >95% lab — Springer 2023). On tabular data, gradient-boosted trees usually edge out RF/SVM (Grinsztajn, NeurIPS 2022) — so XGBoost is a **reasonable prior, not a proven winner for us**.

Model	Fit for Δ (tabular, small)	Interpretability	Cost	Verdict for us
Logistic Regression	Weak–moderate	High	Trivial	**Floor baseline**
Weighted rules (ours)	Good (hand-tuned)	**Highest**	None	**Control** — ML must beat this
Random Forest	**Strong**	High	Low	**Baseline**
XGBoost / LightGBM	**Strong (usually best)**	Med (+SHAP)	Low–med	**Leading candidate — prove it**
SVM (RBF)	Moderate	Low	Med	Secondary baseline
MLP / deep nets	Weak on small tabular	Low	High	**Reject for Tier 1**
Sequence (Transformer/LSTM)	Needs event sequence	Low	High	**Tier 2**

Model	Fit for Δ (tabular, small)	Interpretability	Cost	Verdict for us
Graph (GNN)	Needs data-flow graph	Low	High	**Tier 2 (novelty)**
Anomaly (OC-SVM/IsoForest/AE)	Good for *unknown*	Med	Low–med	**Complementary** (§8)

> **The defensible procedure (this is the answer)** OURS
>
> We do **not** pre-declare a winner. We run a **bake-off** — LogReg (floor), rules (control), RF, XGBoost, SVM — on the **same Δ features**, and **select by PR-AUC and false-positive-rate at fixed recall**, under **time-based, extension-level cross-validation** (§10), reporting accuracy, precision, recall, F1, ROC-AUC, PR-AUC and the confusion matrix, with class imbalance handled and **SHAP** for explanation. The final model is whatever wins — our prior is XGBoost, but it must earn it.

6. What Exactly Will the Model Predict?

```
INPUT     : two versions (v_N trusted baseline, v_{N+1} candidate) of the SAME extension
FEATURES  : Δ = f(T_{N+1}) − f(T_N)  − the 24-dim behavioural-difference vector
            (+ static manifest / permission delta)
MODEL     : the bake-off winner (§5)  →  probability
PREDICTION: P(the update introduced malicious / security-relevant behavioural change) ∈ [0,1]
ACTION    : rank / flag the update for review + explainable evidence (fired rules + SHAP)
```

OURS **What it is not, and cannot establish (state plainly):** it is **not** "is this extension malicious" (single-version), **not** code-vulnerability discovery, **not** zero-day vulnerability finding. It **cannot** prove intent/legality; cannot see payloads that **never trigger** in our run (server-gated/time-bombed); cannot judge behaviours **absent from training** without the anomaly layer (§8); cannot generalise across unseen extension *families* without anti-memorisation care (§10). Output is **decision support**, not an automated verdict.

7. Dataset Strategy — what exists, why we must build our own

Source	Samples	Labels	Versions?	Limits
mallorybowes/chrome-mal-ids	1000s of malicious IDs/IOCs	malicious	No	IDs only, no code/versions
"It's not Easy" ground truth (2025)	341 malicious	malicious	No	Small; single-version
Antivirus-for-extensions set	990 malware / 22 benign *(verify)*	both	No	Tiny benign side; single-version
You've Changed corpus	~922k versions / 6 yr	weak (rating-anomaly)	**Yes**	Static-oriented; weak labels; not a clean download
chrome-stats / crx4chrome	version histories, old .crx	none	**Yes**	Unofficial; integrity caveats; popular only
GitHub open-source extensions	many	benign	**Yes (tags)**	**Best real benign update-pair source**
Web Store crawl	~100k+	none	latest only	No history; unlabeled

No public labelled dataset of malicious update pairs exists OURS

So we **construct our own**: (1) **real benign pairs** — GitHub releases + archive versions + weekly self-snapshots → the false-positive set; (2) **synthetic malicious pairs** — real benign v1 + injected payload families → the detection set; (3) **real anchors** — reconstructed Cyberhaven + archivable IDs from chrome-mal-ids → held-out external validity. This is the honest **synthetic-train / real-validate** design, forced by the evidence.

8. Unknown / Hidden Threats — classification vs discovery

OURS Keep two claims strictly separate: **(A)** classification of *known* malicious behaviour patterns (supervised on Δ) — realistic, our main mode; **(B)** discovery of previously-unknown *vulnerabilities* (zero-days) — **we do NOT claim this**; it is a different research problem.

FINDING For *unseen malicious updates* (payloads not in training), anomaly/novelty methods learn "normal benign-update behaviour" and flag deviation — **One-Class SVM, Isolation Forest, autoencoders** (Deep Learning for Zero-day Malware, ACM CSUR 2023). Because our baseline is the extension's **own prior version**, "normal" is unusually well-defined, which *helps* novelty detection.

- OURS **Realistic:** flag an update whose Δ is **anomalous relative to benign updates** even if the exact payload was never seen — a complementary anomaly layer on Δ.
- OURS **Do not claim:** "our ML finds zero-day vulnerabilities." We detect novel malicious *behaviour change*, not novel code vulnerabilities, and only for behaviours our scenarios **trigger**.

9. Static, Dynamic & Honeytrap Pages — what we actually need

- **Static analysis** — keep a **thin** layer only (manifest/permission diff): cheap, pre-execution. **Not** the core; obfuscation/WASM/dynamic code defeat it.
- **Dynamic execution + instrumentation + runtime monitoring — required.** The only way to see obfuscated/condition-gated payloads and MV3 service-worker exfiltration. Already implemented.
- **Honeytrap / deception pages (Hulk HoneyPages) — partially adopt.** They elicit dormant behaviour by satisfying any selector an extension probes — powerful, but **adaptive pages are non-deterministic**, which fights record/replay. OURS We use **scripted deterministic scenarios + a fixed deception test-site** (fake bank/webmail/ collector) — a lighter, determinism-friendly descendant. Full adaptive HoneyPages = stretch.
- **Taint / data-flow** — stretch (Tier-2 graph); our temporal-adjacency `exfil_flows` is the lightweight stand-in.
- **Hybrid static+dynamic** — the recommended architecture.

> *Verdict: dynamic analysis is **necessary, not optional**; deception content is useful and already present in lightweight form; **full adaptive HoneyPages and full taint are scope-inflating** and belong in the stretch tier.*

10. Evaluation Methodology — defensible, leakage-free

- **Extension-level splitting, never file-level.** All versions/pairs of one extension go entirely into train or test — else the model memorises an extension's style and inflates scores. *The single most important anti-leakage rule.*

- **Time-based split** — train on older pairs, test on newer → real deployment, and exposes **concept drift** (Rosenzweig, TWeb 2025).
- **Class imbalance** — report **PR-AUC** and **FPR at fixed recall**; class weights / stratified CV; never accuracy alone.
- **Zero-day/unknown eval** — hold out entire **payload families**; test whether the anomaly layer still flags them.
- **Cross-extension & cross-browser generalisation** — test on unseen extensions (and, stretch, Firefox).
- **Anti-memorisation** — strip library/vendor identifiers; ensure the model keys on **behavioural change**, not on which library/extension it is.
- **Ground truth** — the fake collector's log confirms what actually exfiltrated, so detection is measured against reality, not only labels.

11. Final Synthesis

1. **Problem validation** — a decade of independent incidents, tens of millions of users, two recurring vectors, both expressed as behavioural change on update.
2. **Technical difficulty** — determinism, false positives, unknown payloads, labelled update-pair data are partially/not solved → genuine research.
3. **Vulnerability taxonomy** — 14 classes; we own the behavioural/capability ones, exclude the static code-vulnerability ones.
4. **End-to-end position** — Detect (late), Block/Disable (vendor/enterprise), **Rollback (not for users)**, Recover (irreversible; service-side only) → value = early detection before propagation.
5. **ML strategy** — predict P(update introduced malicious behavioural change) on Δ; decision-support; clear "cannot" boundaries.
6. **Model selection** — bake-off on Δ, chosen by PR-AUC + FPR under time/extension-level CV; XGBoost is prior, not assumption; seq/graph = Tier-2; anomaly = novelty.
7. **Dataset strategy** — construct our own (real benign + synthetic malicious + real anchors); no public update-pair set exists.
8. **Unknown-threat detection** — anomaly on Δ flags novel malicious *behaviour change*; not zero-day *vulnerability* discovery.
9. **Recommended architecture — hybrid:** thin static + dynamic capture (deterministic scenarios + deception site) + update diff + dual scoring (rules + bake-off ML on Δ) + optional anomaly layer. Not full HoneyPages/taint/GNN in Project-1.
0. **Final recommendation — NARROW + REFRAME, do not reject.** Keep dynamic behavioural update-diffing + determinism + explainability. Drop language-aware analysis, consumer prevention, zero-day vulnerability discovery. Reposition for store/enterprise/researcher. Cite You've Changed (CCS 2020) as the static predecessor we improve on dynamically.

12. Complete Faculty Validation — 12 direct answers

1. **Why important?** Trusted, auto-updating, high-privilege code can turn malicious after every existing check, silently, to millions — repeatedly.
2. **Evidence beyond Cyberhaven?** DataSpii, Nano, Great Suspender, Palant's ~87M-install campaign, 2025 Chrome/Edge spyware, 2025–26 crypto stealers (§1).
3. **Why difficult?** Obfuscation defeats static; dynamic behaviour must be elicited and separated from web noise; update-pair data is scarce; false positives are the real bottleneck (§2).

4. **Vulnerabilities beyond code injection?** Malicious updates, credential/cookie/token theft, exfiltration, permission/API abuse, MV3 SW abuse, hijacking, message-passing/native-messaging flaws, CSP weakening, privacy/fingerprinting (§3).
5. **What happens when a malicious update reaches a user?** It usually executes and exfiltrates before detection; browser-layer damage is irreversible (§4).
6. **Detected/blocked/disabled/reverted?** Detect/block/disable: yes, but vendor/enterprise-side and often late. **Rollback: not for users.** Recovery: service-side credential rotation (§4).
7. **Why ML?** To learn feature combinations/weightings the rules can't tune, and give a calibrated rankable score — **only if it beats the rule baseline** (§5).
8. **Why XGBoost — or another?** XGBoost is the tabular prior, not foregone; settled by a bake-off on Δ measured on PR-AUC/FPR (§5).
9. **What data?** A self-constructed corpus — real benign pairs + synthetic malicious + real anchors — because no public labelled update-pair set exists (§7).
10. **Unseen threats?** An anomaly layer on Δ flags novel malicious behaviour change; we do not claim zero-day vulnerability discovery (§8).
11. **Static / dynamic / honeytrap roles?** Dynamic is the core; static a thin cheap layer; deception a lightweight deterministic test-site; full HoneyPages/taint are stretch (§9).
12. **Strongest defensible version?** Deterministic dynamic behavioural update-diffing, hybrid static+dynamic, dual-scored (rules + experimentally-chosen ML on Δ) with an anomaly layer, explainable, store/enterprise/researcher-positioned (§11).

13. Additional References (Part II)

Malicious Chrome extensions, 75M installs removed. *BleepingComputer*. link

Dangerous Chrome extensions, 87M downloads. *Kaspersky*. link

Millions spied on by malicious Chrome/Edge extensions (2025). *Malwarebytes*. link

18 Malicious Chrome/Edge extensions. *Infosecurity Magazine*. link

mallorybowes/chrome-mal-ids — malicious extension IOC list. *GitHub*. link

Detecting Malicious Browser Extensions by Combining ML and Feature Engineering. *Springer 2023*. link

Deep Learning for Zero-day Malware Detection and Classification: A Survey. *ACM Computing Surveys 2023*. DOI

Grinsztajn et al. Why do tree-based models still outperform deep learning on tabular data? *NeurIPS 2022*. arXiv

Rosenzweig et al. Concept drift in extension detection. *TWeb 2025*. PDF

Towards Browser Controls to Protect Cookies from Malicious Extensions (CREAM). *arXiv 2405.06830*. PDF

Chrome enterprise extension management (blocklist / ExtensionInstallForcelist). *Tech Lockdown*. link

> Part I references (You've Changed CCS 2020, ExtPrivA, Hulk, VEX, MV3 Unveiled, Palant, Sekoia, Arcanum, etc.) are listed in the Part I report and not repeated here.

Prepared for the CSD493 progress meeting · extdrift · Part II · Written to test, not assume, the project's validity. Tag key: FACT = established/vendor-documented · FINDING = from a cited source · ASSUMPTION = reasonable but unverified · OURS = our interpretation.